

Sprawozdanie z projektu

Crypto Explorer

Autor: Yauheniya Drozd
Nr indeksu: 279961
Data: 23 czerwca 2026

1. Cel projektu

Celem było stworzenie aplikacji do analizy rynku kryptowalut z wbudowanym modulem optymalizacji robustnej. Aplikacja pobiera dane historyczne z CoinGecko API, oblicza wskaźniki techniczne i wyświetla je na interaktywnych wykresach. Druga zakładka demonstruje różnicę między klasycznym optimum liniowym a rozwiązaniem odpornym na niepewność parametrów.

Wybór tematu był motywowany chęcią połączenia praktycznego narzędzia dla inwestorów z zagadnieniem matematycznym – optymalizacją w warunkach niepewności.

2. Rozwiązany problem

2.1 Analiza rynku

Aplikacja pobiera dane z CoinGecko REST API (endpoint `/coins/{id}/market_chart`) i oblicza następujące wskaźniki:

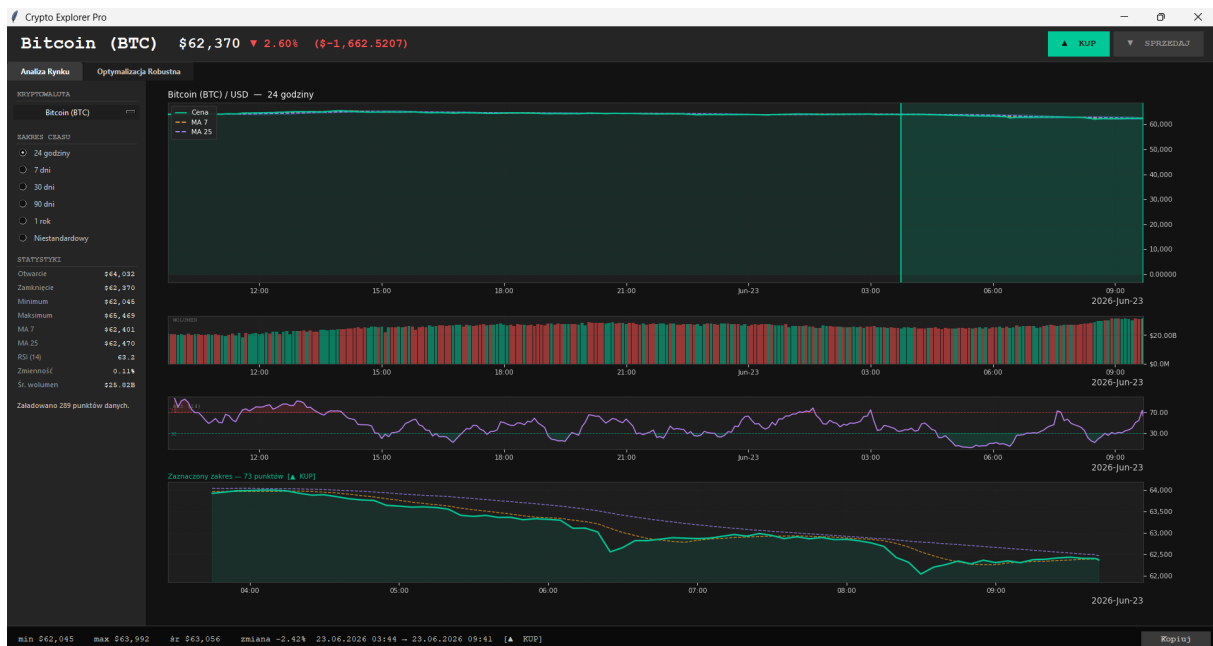
- **MA 7 i MA 25** – średnie kroczące z 7 i 25 obserwacji,
- **RSI (14)** – wskaźnik siły względnej według formuły:

$$RSI = 100 - \frac{100}{1 + \frac{\overline{G}_{14}}{\overline{L}_{14}}}$$

gdzie $\overline{G}_{14}$, $\overline{L}_{14}$ to średnie kroczące zysków i strat z 14 okresów,

- **Wstęgi Bollingera** – $MA_{20} \pm 2\sigma_{20}$.

Użytkownik może wybrać kryptowalutę i zakres czasu, a przesuwając myszą po wykresie – zaznaczyć podzakres i zobaczyć statystyki dla wybranego okresu (min, max, średnia, zmiana procentowa).



Rysunek 1: Zakładka analizy rynku – BTC/USD, zakres 24h. Widoczny wykres ceny z MA 7 i MA 25, pasek wolumenu, panel RSI oraz zaznaczony podzakres z automatyczną statystyką w pasku statusu.

2.2 Optymalizacja robustna

W zakładce *Optymalizacja Robustna* zaimplementowano prosty liniowy problem optymalizacyjny:

$$\max Z = 3x_1 + 2x_2 \quad \text{przy} \quad a_1x_1 + a_2x_2 \leq 30, \quad x_1, x_2 \geq 0$$

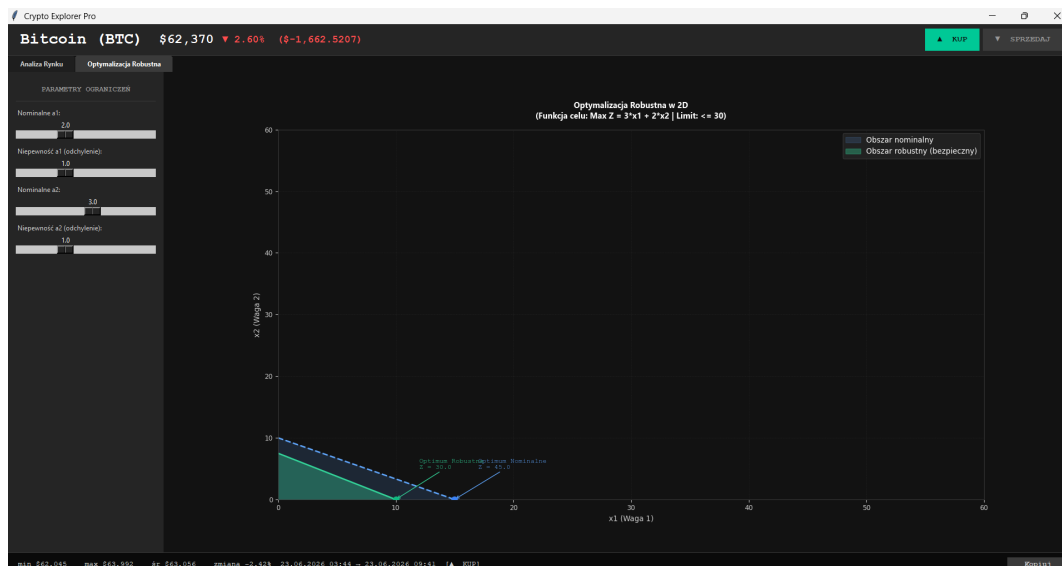
Zmienne x_1 , x_2 to wagi alokacji zasobów do dwóch aktywów. Współczynniki a_1 , a_2 są ustawiane przez użytkownika suwakami, podobnie jak ich odchylenia δa_1 , δa_2 modelujące niepewność.

Optimum wyznaczane jest przez porównanie wierzchołków obszaru dopuszczalnego. Rozwiązanie robustne zakłada worst-case: zamiast a_i używa się $a_i + \delta a_i$, co zawęży obszar dopuszczalny i daje mniejszą, ale *gwarantowaną* wartość funkcji celu.

Dla domyślnych ustawień ($a_1 = 2$, $a_2 = 3$, $\delta a_1 = \delta a_2 = 1$):

- Optimum nominalne: $Z^* = 45$, punkt (15, 0),
- Optimum robustne: $Z^R = 30$, punkt (10, 0).

Różnica $\Delta Z = 15$ to cena odporności – zysk, z którego rezygnujemy w zamian za pewność wykonalności przy perturbowanych danych.



Rysunek 2: Zakładka optymalizacji robustnej. Niebieski obszar – nominalna przestrzeń dopuszczalna, zielony – obszar robustny (bezpieczny). Widoczne oba punkty optymalne.

3. Architektura aplikacji

Aplikacja napisana jest w Pythonie 3.12 z użyciem Tkinter (GUI) i Matplotlib (wykresy). Dane pobierane są przez bibliotekę `requests`, przetwarzane przez `pandas` i `numpy`.

Kod podzielony jest na cztery moduły: `api.py` (klient CoinGecko), `indicators.py` (obliczenia MA, RSI, BB), `config.py` (stałe i kolory), `chart.py` (główne okno aplikacji, 450 linii).

Poniżej fragment odpowiedzialny za wyznaczanie optimum:

Listing 1: Wyznaczanie optimum nominalnego i robustnego

```

1 def get_opt(a, b_val, cx, cy):
2     if cx / a[0] > cy / a[1]:
3         return (b_val / a[0], 0), cx * (b_val / a[0])
4     else:
5         return (0, b_val / a[1]), cy * (b_val / a[1])
6
7 pt_nom, z_nom = get_opt((a1, a2), b, c1, c2)
8 pt_rob, z_rob = get_opt((a1 + da1, a2 + da2), b, c1, c2)

```

4. Wnioski

Projekt pokazał, że nawet prosta implementacja optymalizacji robustnej dostarcza praktycznie użytecznej informacji – w tym przypadku o tym, jak bardzo można ufać wyznaczonemu optimum, jeśli dane wejściowe są niepewne. W kontekście rynku kryptowalut, gdzie zmienność jest bardzo wysoka, podejście worst-case ma bezpośrednie zastosowanie przy zarządzaniu ryzykiem portfelowym.